Nightreign Project Context Handoff

Expanded version with much more detail on the non-prototype scripts, while keeping the prototype sections concise.

Primary scripts covered	Current file to upload to a new chat
Non-prototype renderer	render_nightreign_router_map_coordfix_flags_patched_v12_webp.py
Non-prototype GUI recognizer	nightreign_map_recognizer_gui_hybrid_v15_hotkey_popover_simplified.py
Prototype renderer	render_nightreign_router_map_geomproto_v3_positionanchored.py
Prototype recognizer	recognize_nightreign_map_geometric_proto_v7_stage2dominant.py

This document is meant to be pasted or uploaded alongside the scripts if a new chat needs to continue work without re-discovering the project history. The emphasis here is on the two non-prototype scripts, because those are the ones intended for actual day-to-day use.

1. What a new chat needs to know immediately

- There are two families of scripts. The non-prototype family is the practical workflow in current use: one renderer and one GUI recognizer. The prototype family is a separate experimental branch for geometry-driven recognition.
- The non-prototype renderer is image-generation only. It renders Nightreign maps from the local Nightreign Map Router repository, supports the validated castle fixes, supports Arena Boss title handling, and supports PNG/JPG/WEBP output.
- The non-prototype recognizer is a GUI app. It uses a cache for recognition, uses only all_images/with_text for post-recognition display, supports monitor selection, stable crop logic based on the blue circle, zoom and drag in the main image area, and a configurable Windows global hotkey with a small gear popover.
- The prototype renderer/recognizer are not the same system as the GUI recognizer. They are an experiment based on landmark geometry, not the older full-image comparison workflow.
- If work is about the production renderer or the production GUI recognizer, start from the non-prototype sections of this document. Only switch to the prototype sections if the task is explicitly about the geometric recognition branch.

2. Non-prototype renderer: detailed handoff

Current file: **render_nightreign_router_map_coordfix_flags_patched_v12_webp.py**.

This is the current practical renderer. It is the culmination of the earlier v8-v11 fixes and then a simple v12 addition to support WEBP output. It is the script to use when you want to batch-render all layouts, create recognition reference images, or produce one-off debug outputs.

2.1 What the renderer does

- Reads the local Nightreign Map Router checkout instead of shipping its own map data.
- Loads pattern layout JSONs from reference_material/pattern_layouts.
- Loads the master POI coordinate list from public/assets/maps/poi_coordinates_with_ids.json.
- Loads the layout-specific coordinate JSON for the relevant Shifting Earth.

- Builds dynamic POIs from the pattern file and fixed POIs from the coordinate JSON, then merges and deduplicates them.
- Stitches the 6x6 map tile grid into a single map image.
- Draws icons and, optionally, labels with collision-avoidance behavior similar to the router.
- Saves single-image or batch-image outputs in PNG, JPG, or WEBP.

2.2 Why this script exists instead of using the repo UI directly

The repo is useful for understanding data and logic, but this Python renderer is an offline export tool. It was built because the user needed deterministic batch output, extra CLI controls, and a few corrections that were visually validated outside the repo UI. The most important of those validated fixes are the castle handling and the Arena Boss title handling.

2.3 Important historical fixes already included

Version / theme	What changed and why it matters
v9 Arena Boss fix	Arena_Boss POIs are treated like Field Bosses for label generation and metadata collection, so bosses such as C
v10 castle offset	Restored the hard-coded castle render offsets that were visually validated by the user. This is treated as a require
v11 castle enemy type label	Restored the castle enemy-type label (for example Trolls) so the castle can show three castle-related labels wher
v12 webp	Added WEBP as a valid output format alongside PNG and JPG.

2.4 Core data flow inside the renderer

```
layout JSON  ->  load_dynamic_data()
coordinate JSON ->  build_all_pois()
master POI list ->  resolve canonical coordinates
tile folders    ->  stitch_map_tiles()

merged POIs + map image
        |
        +-> icon_for_poi()
        +-> title_text_for_poi()
        +-> place_titles()
        |
        --> render_layout()
        --> save_image()
```

Important functions to remember:

Function	Role in the pipeline
parse_cli()	Defines all CLI flags, including repo path, layout selection, text mode, naming, extension, zip, and debug opt
load_dynamic_data()	Builds the dynamic POIs and metadata lists from a layout_XXX.json file.
build_all_pois()	Merges dynamic and fixed POIs, deduplicates them, and applies layout-specific exceptions.
icon_for_poi()	Selects the final icon filename, including special-case overrides such as Castle.png for POI 159.
title_text_for_poi()	Computes the label text and priority for each POI. This includes the castle enemy-type label and Arena Boss
place_titles()	Runs the simple collision-avoidance loop for labels.
stitch_map_tiles()	Stitches the 6x6 tile grid into one base map image.
render_layout()	End-to-end render for one layout and one text mode.
save_image()	Writes PNG/JPG/WEBP depending on the chosen extension.

2.5 Key assumptions and special cases

- POI 159 is the castle and is treated specially. It uses Castle.png and also receives the validated render offset.
- Castle render offsets are 50 px on X and 75 px on Y after map scaling.
- Arena_Boss.png is not guaranteed to exist in the public icon folder, so there is a manual fallback to Field_Boss.png.
- Map_Event.png also has a manual fallback to Event.png.
- The renderer uses only the existing five Shifting Earth map layouts from the base repo, not a Great Hollow layout.
- The script is intended to mirror the router pipeline closely, but with explicit patches where the user visually validated that the original behavior was insufficient.

2.6 Command-line behavior the new chat should remember

Use case	Command pattern
All layouts, text on, numbered names, WEBP	python render_nightreign_router_map_coordfix_flags_patched_v12_webp.py --repo ".../nightreign-map
Single layout	python render_nightreign_router_map_coordfix_flags_patched_v12_webp.py --repo ".../nightreign-map
Both text modes	omit --text and let the script create both no_text and with_text folders
Zip outputs	add --zip
Use different start number	add --start-number N

2.7 Practical notes for the new chat

- If the user reports a missing castle label, remember that the castle enemy-type text branch belongs in title_text_for_poi().
- If the user reports an icon showing but no label, check whether the icon family is represented in the metadata collection lists and whether the title logic has a branch for that icon.
- If the user asks for a new output format, save_image() and the --ext choices are the two places to change.
- If the user asks about DLC support on the existing map, there is a separate experimental script for manifest-driven overlays; it is not the same as the main renderer.

3. Non-prototype GUI recognizer: detailed handoff

Current file: **nightreign_map_recognizer_gui_hybrid_v15_hotkey_popover_simplified.py**.

This is the practical GUI recognizer, not the geometric prototype. It is the line that evolved from the earlier capture-and-rank script into a hybrid app that uses a recognition cache, a display image set, a stable crop flow, zoom and drag in the main result view, and a configurable global hotkey on Windows.

3.1 What this GUI is for

- Run recognition from a live monitor while the game is open.
- Show logs in the left pane, auto-scrolling and resetting each run.
- Show one large result image and a right-side strip of the top 5 matches.
- Allow clicking thumbnails to swap the large image.
- Allow zooming and dragging the large image for inspection.
- Allow monitor selection from a dropdown.
- Allow hotkey-based triggering without focusing the app window.

3.2 Required runtime assets

Asset	Expected location / purpose
Recognition cache	nightreign_ref_cache_square_gui.npz; searched near the executable/script and also in all_images/no_text a
Display images	all_images/with_text; this is where the top result image and the top 5 thumbnails come from.
Log file	recognition.log in the runtime directory; overwritten each run.
Settings file	nightreign_settings.json in the runtime directory; stores the configured global hotkey.

3.3 Important constants and defaults

Setting	Current default / meaning
DEFAULT_HOUGH_REGION	right_half
DEFAULT_HOUGH_SCALE	0.5
DEFAULT_PANEL_SCALE	1.12
DEFAULT_SEARCH_RADIUS	80
DEFAULT_SQUARE_RATIO	0.69
THUMB_COUNT	5
Default global hotkey	Ctrl + Alt + M
Display directory	all_images/with_text

3.4 Recognition pipeline in this GUI

```
selected monitor
   -> capture monitor image
   -> detect map circle (blue-ring-first logic)
   -> derive square crop box
   -> cropped_panel
   -> center_square_crop(cropped_panel, 0.69)
   -> preprocess_for_similarity()
   -> preprocess_for_gradients()
   -> phash()
   -> compare_precomputed() against cached references
   -> rank descending by score
   -> resolve top result image paths from all_images/with_text
```

This GUI is still the image-similarity recognizer, not the geometric prototype. The matching stage is cached-array comparison using low-frequency grayscale, gradient magnitude, and a perceptual hash.

3.5 Why the crop logic looks the way it does

- The crop logic went through multiple revisions because even small crop shifts destroyed recognition quality.
- The stable line eventually moved to a blue-circle-aware approach and then to a blue-circle-first practical recognizer family.
- The practical GUI lineage therefore cares a lot about: monitor selection, correct map-panel crop, previewing the cropped panel before ranking, and keeping the crop stable across different monitors.

3.6 UI layout and interaction model

Area	Behavior
Left pane	Recognition log with timestamps, auto-scroll, and reset on each run.
Top bar	Run button, info labels (best/selected/score), gear icon, status badge, and monitor dropdown.
Main image area	Large result image. Supports wheel zoom and click-drag pan. Double-click resets to fit.
Right column	Top 5 match thumbnails stacked vertically with no padding between them. Clicking swaps the main image.
Hotkey popover	Opened from the gear icon; lets the user capture and apply a new global hotkey.

3.7 Hotkey system

- Global hotkeys are implemented for Windows in this build.
- The app ships with Ctrl + Alt + M preconfigured.
- The hotkey can be changed from inside the program through the gear-triggered popover.
- The popover is centered over the recognizer window itself, not the primary monitor.
- The simplified v15 popover intentionally relies on the native window title bar and native close button; redundant in-popover title/close controls were removed.
- If the chosen hotkey cannot be registered because another app owns it, the popover reports failure instead of silently accepting it.

3.8 Monitor selection

The GUI supports multiple connected screens through a dropdown. The selected screen is used for capture instead of always forcing the primary display. This was introduced after earlier versions showed that correct monitor selection matters, but crop correctness matters even more.

3.9 WEBP support and packaging behavior

- Display image loading is WEBP-aware. It tries Pillow first and can fall back to cv2-based loading for WEBP display if needed.
- The GUI can be packaged with PyInstaller while bundling only the cache file and the WEBP files inside all_images/with_text.
- The common packaging pattern is to omit all_images/no_text and rely on the cache plus all_images/with_text only.

```
python -m PyInstaller --noconfirm --clean --windowed --name NightreignRecognizerHybrid \
  --add-data "nightreign_ref_cache_square_gui.npz:." \
  --add-data "all_images/with_text/*.webp:all_images/with_text" \
  nightreign_map_recognizer_gui_hybrid_v15_hotkey_popover_simplified.py
```

3.10 Known strengths and limits

- Strengths: practical UI, stable enough for routine use, strong visual inspection workflow, monitor choice, hotkey trigger, and easy packaging.
- Limit: it still depends on crop accuracy because the core matcher compares cached reference features against the cropped panel region.
- Limit: it is not a semantic recognizer; it does not explicitly reason about landmark geometry the way the prototypes do.
- Limit: global hotkeys are Windows-only in this build.

4. Prototype renderer: concise handoff

Current file: **render_nightreign_router_map_geomproto_v3_positionanchored.py**.

This is the current experimental renderer for the geometric recognizer branch. The prototype detail in the original handoff was already acceptable, so this section stays concise.

- It extends the rendering workflow by exporting recognition metadata rather than only rendered images.
- It exports landmark templates and per-layout normalized landmark coordinates.
- It is meant to support a recognizer that first classifies Shifting Earth, then matches landmark geometry.
- The v3 line is the position-anchored prototype generation line.
- Typical output is a package such as nightreign_geom_proto_v3.npz plus a recognition_metadata_v3 folder.

```
python render_nightreign_router_map_geomproto_v3_positionanchored.py \
  --repo "C:/Users/eduar/Documents/CodeProjects/nightreign-map-router-fork" \
  --text off --name-mode numbered --ext webp \
  --export-recognition-package "nightreign_geom_proto_v3.npz" \
  --export-recognition-dir "recognition_metadata_v3"
```

Prototype landmark set includes the user-selected discriminative classes, including Sorcerer's Rise in the later revisions.

5. Prototype recognizer: concise handoff

Current file: **recognize_nightreign_map_geometric_proto_v7_stage2dominant.py**.

- This is a CLI prototype, not the GUI recognizer.
- It works from a recognition package exported by the prototype renderer.
- The pipeline is: crop panel, classify Shifting Earth, score candidate layouts geometrically, then do a stronger stage 2 ranking.
- v7 specifically reduced STAGE1_KEEP, added stage 2 progress logging, and made the final ranking stage-2-dominant.

```
python recognize_nightreign_map_geometric_proto_v7_stage2dominant.py \
  --package "nightreign_geom_proto_v3.npz" --monitor 1
```

- The geometric prototype remains experimental. It exposed useful structure but also showed that stage 1 could over-credit wrong layouts and that stage 2 could become the new bottleneck.
- Do not confuse prototype results with the production GUI recognizer results; they are solving the same problem with different recognition philosophies.

6. If a new chat needs quick operational context

Paste or summarize the following points:

- There are four relevant scripts: one production renderer, one production GUI recognizer, one prototype renderer, and one prototype recognizer.
- The production renderer is the v12 WEBP script and already includes the important castle and Arena Boss label fixes.
- The production recognizer is the v15 GUI hotkey-popover simplified script. It is the practical recognizer to modify unless the task is explicitly about geometric experimentation.
- The geometric prototype is separate and uses exported metadata plus landmark reasoning; it is not yet the primary workflow.

- If the task is about actual day-to-day use, default to the production recognizer and production renderer.
- If the task is about semantic/landmark-based recognition, default to the prototype pair instead.

7. Recommended upload set for a new chat

Upload	Why
`render_nightreign_router_map_coordfix_flags_patched_v12_webp.py`	Main practical renderer.
`nightreign_map_recognizer_gui_hybrid_v15_hotkey_popover_simplified.py`	Main practical GUI recognizer.
`render_nightreign_router_map_geomproto_v3_positionanchored.py`	Prototype renderer for geometric recognition assets.
`recognize_nightreign_map_geometric_proto_v7_stage2dominant.py`	Latest prototype recognizer in this branch.
This PDF	Human-readable project context and script roles.

If the new chat is meant to work only on the production tools, uploading the first two scripts plus this PDF is usually enough. Upload the prototype pair only if the new task is specifically about the geometry-based recognition branch.